FACILITATOR QUESTION BANK

Day 5 — Anticipated Questions and Model Answers

Doubt clarification for agentic AI, Skills, guardrails and the capstone

Questions participants are likely to raise, with answers written the way you would say them in the room. Difficult and challenging questions are marked, with guidance on handling them without losing the room or overclaiming. The final section covers questions about the week as a whole.

Enterprise AI Engineering Bootcamp · Companion to the Day 5 presentation and study material

A. Agentic Architecture

Q Everyone is building agents. Are we behind if we build workflows?

No. A great deal of what is publicly described as agentic is a workflow with a conditional branch, presented well. The organisations genuinely running autonomous agents in production run them inside tight guardrails, on recoverable actions, with per-step observability and human approval on anything expensive. That is what today covers. Building a reliable workflow now and adding autonomy where it earns its place is not falling behind — it is the sequence that works.

Q How do we decide between a workflow and an agent?

Can you draw the flow chart? If you can, even a large one with many branches, build the workflow. You get the same outcome with a fraction of the cost, latency and failure surface, and you can test it. Agency is warranted when the branching genuinely cannot be enumerated — when the next question depends on the previous answer in a way you cannot list in advance. That is a real category and it is smaller than the vocabulary suggests.

Q Our agent works well in testing but fails in production. Why?

Usually the compounding arithmetic. In testing you try the paths you thought of, which are the ones the agent handles. In production it encounters combinations you did not anticipate, and each additional step multiplies the failure probability rather than adding to it. Instrument per-step success rate and you will almost certainly find one or two weak steps dragging the whole chain down. Fix those, or make them deterministic, before touching anything else.

Q How many steps is too many?

Work backwards from your reliability target. If you need 80% end-to-end and each step is 95% reliable, you have about four steps of budget. If each step is 90%, you have two. That arithmetic is usually more sobering than any general guidance, and it is the right way to have the conversation. Then look at which steps can be made deterministic — every one you convert to code comes out of the probabilistic chain entirely.

Q Where should we put approval gates?

On irreversibility, financial threshold, safety relevance and external visibility. Not on model confidence, which is not a safety property and is frequently miscalibrated — a Day 1 point that matters here. And design the gate so it is genuinely usable: the approver needs the proposed action, the reasoning, the evidence, and one click. A gate that requires reconstructing the agent's reasoning from logs will be rubber-stamped within a fortnight, and that is worse than no gate because it looks like control.

Q Should the agent remember previous conversations?

Distinguish state from memory first. State is this task — keep it structured, small, and owned by your code, not accumulated in a prompt string. Memory is across tasks, and it is data: it needs retention rules, access control and a refresh policy like anything else. Retrieve memory when relevant rather than carrying it, or your context grows every turn, your cost rises, and the lost-in-the-middle effect from Day 3 starts degrading attention to your own instructions.

Q How do we test an agent when it behaves differently every run?

Three levels. Unit-test the deterministic components, which should be most of them if you followed Module 1.5. Scenario-test the agent on a fixed set of tasks with known correct outcomes, run several times each, and report the success rate with a confidence interval — Day 1's machinery applies directly. And instrument per-step success in production, because that is the only place you see the paths you did not think of.

B. Agent-to-Agent and Skills

Q When should a capability be an agent rather than a tool?

Prefer tools. Every capability expressed as a tool removes a source of non-determinism, becomes unit-testable and has a predictable cost. Reach for an agent only when the sub-task genuinely requires multi-step reasoning that cannot be enumerated — and when it does, give it the narrowest possible goal. Most sub-agents in enthusiastic designs are tools with extra steps and extra failure modes.

Q Is a Skill just a prompt template with extra process?

A prompt is one of its components. What makes it a Skill is that it is packaged, versioned, tested, owned and governed — instructions plus schemas plus tool bindings plus safety rules plus a regression suite. The practical test: can another team use it without talking to you, and will you know if a change breaks it? A prompt template fails both. That difference is exactly what makes reuse safe rather than merely convenient.

Q What should our first Skill be?

Something recurring, boring and well understood, where consistency matters and the cost of doing it inconsistently is real. A troubleshooting sequence for one product family, or a retrieval Skill applying your access rules, version filtering and citation requirements consistently. Do not start with something novel or something that takes actions — start with something you already do the same way twenty times a month and package it properly.

Q Who owns the Skill registry?

Someone must, or it decays into a liability within a year. In practice a platform or enablement function owns the registry itself — the standards, the review process, the deprecation policy — and individual teams own the Skills they publish. What does not work is a registry anyone can publish to with no review, which produces a directory of half-working Skills nobody trusts, and everyone goes back to copy-pasting prompts.

Q How heavy should the approval process be?

Proportionate to reach. A Skill that only reads and summarises needs a light review — correctness and a test suite. A Skill that can order parts or close a ticket needs authorisation review, a red-team pass, and an audit trail. Uniform heavy process kills adoption and drives people to the copy-paste approach you were trying to prevent, so the tiering matters as much as the review itself.

Q What is the difference between a Skill and an MCP server?

An MCP server exposes capabilities over a protocol — it is an integration boundary, typically owned by the team that owns the underlying system. A Skill is a packaged way of performing a task, which may bind to tools from one or several MCP servers. One is about access; the other is about know-how. A troubleshooting Skill might call three tools from two MCP servers and add the instructions, safety rules and test cases that make the sequence correct for your organisation.

C. Guardrails and Security

Q Is prompt injection a real risk for an internal system?

Yes, and internal systems are frequently more exposed rather than less, because the content is trusted by default. Anything your system reads and did not author is a vector — a service ticket a customer filled in, a supplier's technical document, an email, a filename. The relevant question is not whether the source is internal but whether the content was authored by someone you control and whether the system can take an action on the basis of it.

Q Can we not just instruct the model to ignore instructions in retrieved content?

You can, and it helps somewhat, and it is not a control. Your instruction and the attacker's instruction arrive in the same channel, and a sufficiently emphatic or well-placed one can override yours. The architectural defence is that no capability is authorised by anything the model read — authorisation comes from who the caller is, enforced in the orchestration layer outside the model. Treat the prompt instruction as defence in depth, not as the defence.

Q Where exactly should guardrails live?

In the orchestration layer, as code. A guardrail expressed as an instruction is a request the model may decline under pressure; a guardrail expressed as code is a control that runs regardless. This is exactly the same argument as Day 2's point that access filtering belongs in the retrieval query rather than in output redaction — it is the same mistake in a different place, and it is the most common architectural error in guardrail design.

Q How do we stop the system leaking data it retrieved?

Do not let it retrieve it. That is Day 2's secure retrieval: access rights are part of the query, so restricted documents are never candidates. Output scanning is a useful second layer and a poor first one, because by the time content is in the context it has already influenced the answer. If you take one thing from the security material across the week, take that ordering.

Q Our red team exercise found lots of problems. Is our system unusually bad?

No — that is the normal result, and finding them today is the successful outcome. Most teams succeed on boundary testing and access probing within ten minutes. The useful part is the triage: for each success, was the control missing entirely, in the wrong place, or expressed as a prompt instruction rather than as code? Those are three different fixes, and the third is by far the most common.

Q How often should we red team?

Before any significant release, and on a schedule regardless. But the more important shift is from event to suite: every successful attack becomes a permanent safety test case that runs in your regression suite from then on, exactly as production failures became regression cases on Days 1 and 3. Within a year the suite encodes everything anyone has managed to break, and no change can silently reintroduce a past vulnerability.

Q What about cost attacks? That seems minor.

It is minor until someone crafts an input that induces a twelve-step chain with repeated retrieval at every step, and does it a thousand times. Per-caller budgets, step limits and cost alerts take an afternoon to implement. Without them, a single malformed or malicious input can produce a bill nobody authorised, and in an agentic system the amplification factor is much larger than in a simple request-response one.

D. The Capstone

Q We do not have all eighteen items. What do we do?

Mark the gap honestly and state the plan for closing it. A capstone that says "we have no evaluation set; building one is the first thirty days" is considerably stronger than one that invents a number. Diagnostic honesty is one of the four things this is judged on, and it is the one that will matter most when you present this internally.

Q Our baseline is embarrassing. Should we present it?

Yes, and lead with it. A team presenting 0.61 with a clear diagnosis and a ranked improvement plan is in far better shape than one presenting 0.85 with no idea where it came from. Your management will respect the first more once they understand the difference — and the second version tends to collapse the first time someone asks how it was measured.

Q How do we answer the 95% question in the capstone?

With the structure from Module 6.2. Name the specific metric. Give the measured baseline with its interval and the evaluation set. State the realistic ceiling — your inter-annotator agreement or the aleatoric limit. List the ranked interventions with expected deltas and the evidence behind each. Give the cumulative projection as a range. Then state what remains and what would be required to close it. That answers the question honestly and is far more useful than a promise.

Q What if our conclusion is that the use case is not viable?

Then say so, with the evidence, and propose the rescope. That is a legitimate and valuable outcome — it redirects effort that would otherwise be spent for another year. The strongest version names what would make it viable: different ground truth, a narrower task definition, a human-in-the-loop design, or better capture conditions. A well-argued "not as scoped, but here is what would work" is a better result than an optimistic plan nobody believes.

Q Our management will want model work in the first thirty days, not measurement.

Prepare for that conversation explicitly, because it will happen. The argument that works: without a baseline you cannot demonstrate that the model work helped, so month one of model work produces changes of unknown effect. Two to three weeks of measurement makes every subsequent month's work provable. Frame it as the thing that lets you report progress credibly rather than as a delay — and offer one cheap win in parallel, usually threshold tuning, which needs no new data.

E. Difficult Questions — Handle With Care

> **These challenge the premise of the day or the week. Answer honestly rather than defensively.**

Q You have spent the day arguing against agents. Why teach them at all?

Because some problems genuinely need them, and because you will be asked to build them whether or not they are warranted. The argument is not against agency — it is against agency chosen by default. If you can articulate why a workflow will not serve, you should build the agent, and you now know what controls it needs. The teams that get into trouble are the ones who never asked the question.

Q This week has been mostly about process. Where is the actual AI engineering?

A fair challenge. The honest answer is that the model work is the part you already do well — everyone here can train a model or build a retrieval pipeline. What stalls proofs of concept is not modelling skill; it is measurement, diagnosis, architecture, security and the loop that sustains improvement. The week deliberately spent its time on what was missing rather than on what you already have. If you had turned out to be weak on modelling, this would have been a different course.

Q How much of this will we actually implement?

Realistically, some of it, and which parts matters more than how many. If each team leaves with an evaluation set, a failure taxonomy and one measured improvement, the week has paid for itself several times over. The capstone plan is deliberately structured so the first thirty days are achievable by one person — that is the part to hold yourselves to. The rest can follow as the case for it becomes obvious.

Q Our organisation moves slowly. Most of this needs approvals we will not get.

Separate what needs approval from what does not. Building an evaluation set, running a failure taxonomy, tuning a threshold, and writing a labelling guideline need no approval from anyone. Identity integration, data residency decisions and audit retention do — start those conversations now, in parallel, because they run on their own timescale. The mistake is treating the approvals as a gate before any work, when most of the valuable work is not gated at all.

Q Will any of this still be correct in a year? The field moves quickly.

The tools will change and most of the principles will not. Measure before improving, diagnose before collecting, separate retrieval from generation, authorise outside the model, treat model output as untrusted, and write down what you learned — none of those depends on a model generation or a framework. The specific advice most likely to date is in the tooling and the model-selection guidance. The discipline is what you should expect to still be using.

Q What should we do first on Monday?

One thing, and make it the evaluation set if you do not have one. Fifty questions or images with verified correct answers, taken from real usage rather than invented. It is two or three days of one person's time, it makes every subsequent change measurable, and it is the prerequisite for almost everything else this week covered. If you already have one, run the failure taxonomy on a hundred real failures instead.

> **General principle for the final day.** Where the honest answer is "start smaller than we have described", say so. The week's credibility rests on the teams believing the plan is achievable. An ambitious plan nobody starts is worth less than a modest one they finish.

F. Quick Reference — One-Line Answers

If asked	Say
Workflow or agent?	Can you draw the flow chart? Then build the workflow.
How many steps is too many?	Work backwards from your reliability target and the per-step rate.
Where do approval gates go?	On irreversibility and cost. Never on model confidence.
Agent fails in production?	Instrument per-step success. One or two weak steps usually explain it.
Tool or sub-agent?	Prefer tools. Most sub-agents are tools with extra failure modes.
Is a Skill just a prompt?	A prompt is one component. Packaging, testing and ownership make it a Skill.
Skill or MCP server?	MCP is about access. A Skill is about know-how.
Is injection real for internal systems?	Yes. The question is who authored the content, not where it sits.
Can we prompt our way out of injection?	No. Your instruction is in the same channel as the attack.
Where do guardrails live?	In the orchestration layer, as code. Not in the prompt.
Stop data leakage?	Do not retrieve it. Output scanning is the second layer, not the first.
Red team found lots of holes?	That is the normal and successful result. Triage each one.
Missing capstone items?	Mark the gap and state the plan. Honesty scores higher than invention.
Baseline is embarrassing?	Lead with it. A diagnosed 0.61 beats an unexplained 0.85.
Use case not viable?	Say so with evidence, and name what would make it viable.
What first on Monday?	The evaluation set. Everything else depends on it.

Companion documents: **Day 5 Presentation** (42 slides with speaker notes) and **Day 5 Study Material** (concept explanations with analogies and technical depth).